



FEATURE MODEL VISUALIZATION: PROBLEMATIC FEATURE MODEL DIAGNOSTICS

Bachelor's Project Thesis

Reinder Kas, s3189953, r.kas@student.rug.nl,

Supervisor: Dr. H.A. de Weerd

Abstract: Feature Models are a representation of a product within the context of a Constraint Satisfaction Problem. If a solution to a specific product configuration exists, a solver will be able to find it. If there is no solution, it is usually unclear where the problem exactly lies. The bigger the problem, the more Variables and Constraints are present, and the more complex this puzzle of finding the cause of the problem will be. Lots of time can be spent wasting on figuring out the problem. By creating a tool that visualizes where the problem within the Feature Model lies, a lot of problems become visible in the blink of an eye. By comparing the time to diagnose a problematic constraint with this tool and diagnosing the problem from the model definition from a text file, it is clear that this tool gives the user more and better feedback as to where the problem lies and why it is a problem in the first place. By using this tool a lot of unnecessary wasted time can be prevented and users should have a better user experience when creating Feature Models.

1 Introduction

Within the field of Artificial Intelligence and Computer Science, there are many complex real-world problems that researchers try to solve. Many of these problems can be represented intuitively as Constraint Satisfaction Problems (e.g., scheduling or timetable problems). These problems consist of a set of Constraints given by the environment, or model, that the problem tries to emulate. These Constraints can be in many forms; from logical requirements to resource limitations (mostly used for optimization problems).

This paper will focus only on Constraint Satisfaction Problems that deal with the form of logical requirements called Feature Models and tries to tackle a real-world problem that the company Elfsquad has encountered a lot while growing and scaling the application during the last few years of its existence.

1.1 Elfsquad

Elfsquad is a company that specializes in online Product Configurations. The basic principles that satisfy most requirements for their customers con-

sist of constraints that are only logical constraints. After having developed their own software that allows customers to create their own Feature Models (further explained in 1.4), it became apparent that these models become complex very quickly.

The typical Feature Model within the Elfsquad software consists of 2.000 Features and 2.500 constraints (upwards to 12.000 Features and 25.000 constraints). Their customers often have problems with their Feature Models, where these models do not have a solution, meaning their model is unsatisfiable thus they cannot use Elfsquad's software to sell their product. This is a massive problem for customers because not being able to configure a product means a loss of revenue with the wasted time that could have been spent selling a product. In a lot of these cases, the customers call Elfsquad support to troubleshoot their problems, which in turn now also costs Elfsquad extra resources that could and should have been avoided in the first place.

As of right now, Elfsquad's support spends too much time diagnosing Feature Models that do not have a solution. The main problem here is that these models that do not have a solution should

not have been created in the first place. If they are created by customers, then there should exist a tool that empowers the customer to self-diagnose the problem within their own Feature Model. If this tool is easy enough to use and shows the correct amount of information, then the amount of time for the Elfsquad support will spend diagnosing Feature Models should greatly reduce, giving them more time to help with different matters.

Visualization tools already exist for Feature Models, such as FeatureIDE (Thüm et al., 2014), but these visualization tools do not give enough functionality for the Elfsquad company to integrate with. Elfsquad has created its own method of creating Feature Models through its in-house developed Archer Modeling Language. This Archer Modeling Language is more expansive than the well-known Feature Model constraints that FeatureIDE relies on to function. This means that a different solution must be found.

1.2 Archer

Elfsquad has developed its own Modeling Language, known as Archer Modeling Language. This language takes a String-based input and generates a Microsoft Z3 Solver object containing the problem definitions as per the input string.

Since this language is a String-based language, there is no visual representation of what the logic within a Feature Model means. Even smaller Feature Models could become problematic to diagnose since all information describing the model is written in a textual format.

Because other visualization tools do not have the functionality to support the Archer Modelling Language, it has been decided to create an in-house developed tool to give a visual way to represent these models. This thesis functions as the basis of this tool to diagnose problematic Feature Models upon which will be further researched and developed in the future.

1.3 Constraint Satisfaction Problem

A Constraint Satisfaction Problem is a mathematical problem (model) that consists of three components X, D, and C: (Russell & Norvig (2010))

X is a set of variables, $\{X_1, ..., X_n\}$.

D is a set of domains, $\{D_1, ..., D_n\}$.

C is a set of constraints that specify allowable combinations of values.

If a model is created with these three components then a Solver will be able to find a solution to the problem as long as one exists. Many of these solvers exist already, such as Microsoft Z3 Solver (de Moura & Bjørner, 2008), Google OR-Tools (Perron, 2011), or choco-solver (Prud'homme et al., 2019). Each of these solvers works in a different way but should ultimately come to the same solution if the problem is defined correctly.

The problem that this thesis is trying to tackle is that there can also be cases where such a model does not have a solution:

$$\begin{aligned} X &= \{A\}; \\ D &= \{A = \{True, False\}\}; \\ C &= \{(A \neq A)\}; \end{aligned}$$

The solution to this defined model should satisfy the constraint $A \neq A$. Since we know that a Variable can have only a single value assigned to it at a given moment, it is immediately clear that there will be no solution to this model. We cannot assign Variable A such that it is not equal to its own value. This means that the defined model is unsatisfiable.

In this small example, it is clear where the fault in the model lies. There is only a single constraint that we impose on the problem. To make the problem satisfiable again we can either change or remove this constraint. If the model were to be bigger, finding out where the problem is within the logic of the model would become more difficult. More constraints means more complexity and more possibilities of conflict within the model's logic.

The specific form of complex Constraint Satisfaction Problem that this thesis deals with is called a Feature Model.

1.4 Feature Model

Within the field of Software Development there is a compact way to represent all the products of the Software Product Line, called a Feature Model. (Lee et al., 2002)

This Feature Model is a visual way of modelling all the Relationships (Constraints) and Features (Variables) within a product, giving an overview of the

structure dependencies and what is and is not possible within every aspect of a product.

Within this Feature Model, a Feature can be either selected (True) or not selected (False). Other values of the domain such as numeric values are also possible but are beyond the scope of this research paper. At the base of every Feature Model, there is a Root Feature. This is the Feature at the top of the Hierarchical tree, defining the starting point of the Feature Model and the highest point of abstraction for the Model. We define the Root Feature to always be True, since the Root Feature being False would mean that this Feature is not present, thus making the entire Feature Model obsolete.

These Feature Models also have a predefined set of Constraints, typically divided into two relationship types (shown in Table 1.1): the Hierarchy relationships, and the Cross-Hierarchy relationships, sometimes also known as the Parent-Child relationships and the Cross-Tree relationships (Benavides et al., 2010). The definition of these Feature Model Relationships between Parent P and Child C (A and B for Cross-Tree relationships) are given as follows: (Hotz et al., 2014a)

Constraint	Semantics
Mandatory	$P \leftrightarrow C$
Optional	$P \rightarrow C$
Or	$P \leftrightarrow (C_1 \vee C_2 \vee \dots \vee C_n)$
Alternative	$(C_1 \leftrightarrow (\neg C_2 \wedge \dots \wedge \neg C_n \wedge P))$ $\wedge (C_2 \leftrightarrow (\neg C_1 \wedge \dots \wedge \neg C_n \wedge P))$ $\wedge \dots$ $\wedge (C_n \leftrightarrow (\neg C_1 \wedge \dots \wedge \neg C_{n-1} \wedge P))$
Requires	$A \rightarrow B$
Excludes	$\neg A \vee \neg B$

Table 1.1: Feature Model relationships (Hotz et al., 2014a) showing the semantics of the Hierarchical relationships (white background) and the Cross-Hierarchy relationships (light-gray background).

1.4.1 Hierarchical relationships

The Hierarchical relationships (or Parent-Child relationships) are the constraints that define the structure and hierarchy within the model. Since these constraints are defined in this hierarchical

structure, we know that the Parent Feature must also be selected if a Child Feature is selected.

Four main relationships define this hierarchical structure within a Feature Model. These will be defined with examples taken from the Phone Feature Model shown in Figure 1.1. This Feature Model gives a compact and intuitive representation of a Mobile Phone such that the underlying relationships between all the Features within the Feature Model are visible in a visual representation.

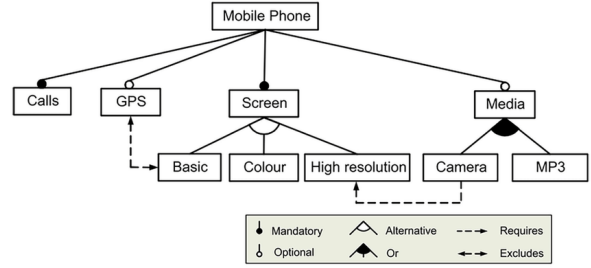


Figure 1.1: Feature Model representation of a simple phone (Benavides et al., 2010).

Mandatory

The Parent and Child must have the same value (e.g., if the Feature **Mobile Phone** is selected, then so is **Screen**).

Optional: The Child can only be selected iff the Parent is selected (e.g., **GPS** can only be selected iff **Mobile Phone** is selected).

Or: There must be at least one Child selected iff the Parent is selected (e.g., if **Media** is selected, then either **Camera**, **MP3**, or **both** can be selected).

Alternative: There must be exactly one Child selected iff the Parent is selected (e.g., if **Screen** is selected, then either **Basic**, **Color**, or **High Resolution** can be selected, but not multiple at the same time).

1.4.2 Cross-Hierarchy Relationship

The Cross-Hierarchy relationships are relationships that are typically inclusion or exclusion statements. These relationships make it possible to define dependencies between Features throughout the whole hierarchy of the tree.

Requires: If Feature A is selected, then Feature B must be selected as well (e.g., if **Camera** is selected, then **High Resolution** must also be selected).

Excludes: Both Features A and B cannot be selected at the same time (e.g., if **GPS** is selected, then **Basic** cannot be selected and vice versa).

At first glance, it might seem confusing that the Optional and Requires constraints have the same implementation, but it is important to separate these definitions. Making a separation between this Parent-Child relationship and the Cross-Tree relationship makes it easier for human users to show and define Hierarchy in a model. Without this distinction, the hierarchy of a model will be ill-defined. Secondly, when diagnosing a Feature Model it will be important to make a distinction between these Hierarchical and Cross-Hierarchy relationships. This importance will be made clear later in section 2.2.3.

2 Method

2.1 Application

The visualization tool consists of two separate applications: the front-end and the back-end. These are built using a Client-Server pattern and during the development process will run locally in Docker. The API is the Server that provides service and the interface is the Client requesting services. This pattern is chosen because it fits well with the existing implementations and philosophies of Elfsquad (API-first design).

The complete application’s source code is stored in a Github repository. This ensures that all changes to the code can be checked and approved and the complete history of the development of the tool is visible. Permission to access the source code regarding this thesis can be provided via a personal email if approved by Elfsquad.

2.1.1 Interface

The interface is created with AngularJS. This is a TypeScript-based, free and open-source single-page web application framework. This framework is also what Elfsquad uses for their front-end and allows for making components that can later be directly

implemented into Elfsquad’s source code without extra work. The visualization within the tool makes use of the D3 JavaScript library (Rothenhäusler, 2022) which is meant to be used for dynamic, interactive data visualizations in web browsers. This library is perfect for visualizing the complex structure of the Feature Models.

2.1.2 API

The API is created in .NET 7.0. Since the Archer Modeling Language makes use of the Microsoft Z3 Solver it is best to use the Z3 solver for this diagnostics tool as well. Upon initializing a new Z3 solver object, it is possible to set flags that determine the behaviour of the Z3 Solver. The two flags most relevant for what this tool wants to achieve are the flags **proof**, and **unsat_core**.

The **proof** flag tells the Z3 Solver to generate a proof in Lisp showing the internal logical structure of the Z3 Solver’s model as to why it could not satisfy the given constraints. The **unsat_core** flag tells the Z3 Solver to generate the minimal set of constraints that is still unsatisfiable (Unsatisfiable Core).

This tool will have multiple functionalities. It will show the proof generated by the Z3 solver in Lisp and it will also generate a list of constraints that have to be removed to make the problem satisfiable again according to the Fast Diagnose algorithm (Hotz et al., 2014b). The tool will also produce a visual representation of the entire given Feature Model and highlight the constraints that are present within the Unsatisfiable Core and Fast Diagnose result. This shows the end-user where the problem in the model is in a graphical way.

2.2 Design Choices

2.2.1 Proof

The initial research was done into making use of the Proof generated by the Microsoft Z3 solver in order to decipher why a Feature Model does not have a solution. After implementing the Z3 solver and generating this proof, it quickly became clear that it is not a feasible method of finding the solution to a problematic Feature Model for this thesis. A small example Feature Model (hereafter refer-

enced as Simple Model) is defined as:

$$\begin{aligned} \text{Root} &\xrightarrow{\text{Mandatory}} \text{Child} \\ \text{Child} &\xrightarrow{\text{Excludes}} \text{Child} \end{aligned}$$

Or in the CSP notation:

$$\begin{aligned} X &= \{\text{Root}, \text{Child}\}; \\ D &= \{\text{Root} = \{\text{True}\}, \\ &\quad \text{Child} = \{\text{True}, \text{False}\}\}; \\ C &= \{(\text{Root} == \text{Child}), \\ &\quad (\neg \text{Child} \vee \neg \text{Child})\}; \end{aligned}$$

Here we have the situation where the Root Feature must be selected. The Mandatory constraint implies that the Child must also be selected. The Exclude constraint implies that if the Child Feature is selected, then the Child Feature cannot be selected. This means that there is no way to assign the Child Feature a value such that it will satisfy all constraints present in the problem set.

The intuitive method of resolving this problem is to remove the Exclude constraint. The proof generated in Lisp by the Z3 solver as seen in Figure 2.1 does not show this intuitively at all and is significantly complex already.

```
(let ((a1 (def-axiom (or (not (= Root Child)) (not Root) Child)))
      (a2 (monotonicity (rewrite (= (= Root true) Root))
                        (= (=> proxy12 (= Root true)) (= proxy12 Root))))
      (a3 (rewrite (= (=> proxy12 Root) (or Root (not proxy12))))))
      (a6 (asserted (=> proxy11 (or (not Child) (not Child)))))
      (a7 (rewrite (= (or (not Child) (not Child)) (not Child))))
      (a8 (= (=> proxy11 (or (not Child) (not Child)))
              (= proxy11 (not Child))))
      (a9 (rewrite (= (=> proxy11 (not Child)) (or (not Child) (not proxy11))))
      (a10 (= (=> proxy11 (or (not Child) (not Child)))
              (or (not Child) (not proxy11))))
      (a12 (rewrite (= (=> proxy10 (= Root Child))
                      (or (not proxy10) (= Root Child)))))
      (let ((a4 (trans a12
                      a13
                      (= (=> proxy12 (= Root true)) (or Root (not proxy12)))))
            (a11 (unit-resolution (mp a6
                                     (trans (monotonicity a7 a8) a9 a10)
                                     (or (not Child) (not proxy11)))
                                (asserted proxy11)
                                (not Child)))
            (a13 (mp (asserted (=> proxy10 (= Root Child)))
                    a12
                    (or (not proxy10) (= Root Child)))))
      (let ((a5 (mp (asserted (=> proxy12 (= Root true)))
                    a4
                    (or Root (not proxy12)))))
      (unit-resolution a11
                      (unit-resolution a5 (asserted proxy12) Root)
                      a11
                      (unit-resolution a13 (asserted proxy10) (= Root Child)
                      false))))
```

Figure 2.1: Proof generated by the Microsoft Z3 solver for the Simple Model.

The size and complexity of this proof only increase as the complexity of the Feature Model increases as well (more models with their proofs depicted in appendix A.2). Therefore the decision has been made to research other options for diagnosing the Feature Models first.

2.2.2 Unsatisfiable Core

The next point that was researched was how to make use of the so-called Unsatisfiable Core. This Unsatisfiable Core is the minimal set of constraints in the Feature Model that produce an unsatisfiable result. This minimal set is thus guaranteed to contain the problem of the Feature Model and can be seen as the focus point for diagnosing the problem within the unsatisfiable model as seen for the Simple Model in Figure 2.2.

Constraints in Unsatisfiable Core:

(= Root Child)

(or (not Child) (not Child))

(= Root true)

Figure 2.2: Unsatisfiable Core generated by the Microsoft Z3 solver for the Simple Model.

This generated Unsatisfiable Core is already a way more uncluttered way of indicating where the problem in the model lies. However, there are two problems with this solution.

Firstly, the Z3 result is still written in Lisp. Lisp is a language that the end users are most likely not familiar with. The customers of Elfsquad themselves will ideally use this tool. Since Elfsquad's software is intended to have a low threshold of technical knowledge, this does not suit the philosophy of Elfsquad's software.

Secondly, while this result reduces the problem to a minimum conflict set, it does not return all the problems within the Feature Model. It only gives back the smallest subset of the original model definition that still contains a problem as shown in Figure 2.3.

```

model [Phone] {
  [Phone] - mandatory -> [Screen]
  [Phone] - mandatory -> [FingerPrint]
  [Phone] - mandatory -> [GPS]
  [Phone] - mandatory -> [Media]

  [Screen] - excludes -> [FingerPrint]
  [FingerPrint] - excludes -> [GPS]
  [GPS] - excludes -> [Media]
  [Media] - excludes -> [Screen]
}

```

Constraints in Unsatisfiable Core:
(= Phone Screen)
(= Phone Media)
(or (not Media) (not Screen))
(= Phone true)

Figure 2.3: Unsatisfiable Core generated by the Microsoft Z3 solver for a Feature Model with the same Excludes constraint highlighted in both the model definition and the Unsatisfiable Core result.

In the case of Figure 2.3, the highlighted constraints is a single excludes relationship that is present in the Unsatisfiable Core. This is because this singular relationship already causes the model not to be satisfiable. An ideal solution would be to show that all four excludes relationships cause problems within this Feature Model.

These discussed issues with presenting the information this way means that the solution needs to be more intuitive than just showing the Unsatisfiable Core.

2.2.3 Fast Diagnose

To be able to give the end-user information about how to correctly change the Feature Model such that it becomes satisfiable again, the Fast Diagnose algorithm (Hotz et al., 2014b) is implemented as seen in pseudo-code in algorithm 2.1.

This algorithm is based on the fact that it is impossible to build an unsatisfiable Feature Model with Hierarchical constraints alone (which builds abstraction but not complexity). The output of the

algorithm is a list of Cross-Hierarchy constraints that need to be removed to make the Feature Model satisfiable again.

This Fast Diagnose algorithm is a divide-and-conquer algorithm. It takes both the Cross-Hierarchy constraints (C) and all the constraints present in the Feature Model (AC) and tries to find the Cross-Hierarchy constraints that cause the Feature Model to be unsatisfiable. By dividing these Cross-Hierarchy constraints (C_1, C_2) and systematically removing these from the constraints that are checked to be satisfiable, we end up with the constraints that are the cause of trouble within the Feature Model.

For example; if AC is unsatisfiable, but $AC - C_1$ is satisfiable, then we know that the problem in the model lies within the C_2 set of constraints. Since this set is unsatisfiable, through recursion this same logic is applied again. In the next iteration of the recursion, the variable AC is now taken as $AC - C_2$ (which is the same as $AC \cup C_1$), C becomes C_2 and now divide this newly given C up into two new sets (C_1, C_2). This recursion is applied until one or more problematic constraints are found and we return these constraints as a solution.

Algorithm 2.1 The Fast Diagnose algorithm (Hotz et al., 2014b)

```

1: function FASTDIAGNOSE( $C \subseteq AC, AC = \{c_1..c_t\}$ )
2:   if (ISEMPTY( $C$ ) || !HASOLUTION( $AC - C$ )) then
3:     return  $\emptyset$ 
4:   else
5:     return FD( $\emptyset, C, AC$ )
6: function FD( $D, C = \{c_1..c_q\}, AC$ )
7:   if ( $D \neq \emptyset$  && HASOLUTION( $AC$ )) then
8:     return  $\emptyset$ 
9:   if SINGLETON( $C$ ) then
10:    return  $C$ 
11:    $k \leftarrow \frac{C}{2}$ 
12:    $C_1 \leftarrow \{c_1..c_k\}$ 
13:    $C_2 \leftarrow \{c_{k+1}..c_q\}$ 
14:    $D_1 \leftarrow$  FD( $C_2, C_1, AC - C_2$ )
15:    $D_2 \leftarrow$  FD( $D_1, C_2, AC - D_1$ )
16:   return  $D_1 \cup D_2$ 

```

Once the minimal conflict set is obtained from this algorithm, the tool can present the resulting constraints to the end user to inform them which constraints are the exact constraints that cause problems with the Feature Model. The user then has the option to either change or remove these Constraints.

Remove Constraint to make Satisfiable:

[FingerPrint] - excludes -> [Screen]

[FingerPrint] - excludes -> [GPS]

[GPS] - excludes -> [Media]

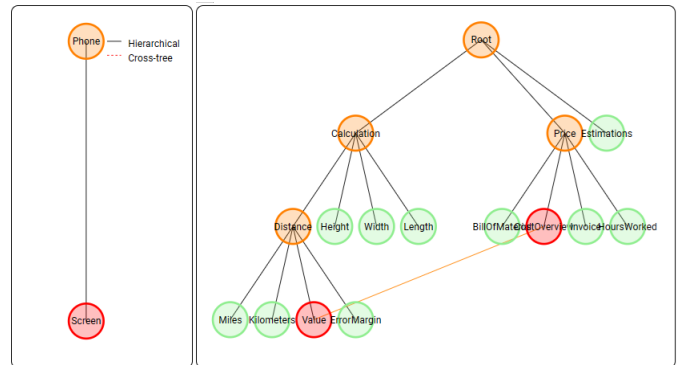
[Media] - excludes -> [Screen]

The notation in Figure 2.4 differs from Figure 2.3 because the Fast Diagnose algorithm only uses the Z3 solver to check if the current set of constraints is satisfiable. Since we still have the notation of the Archer language before that, it has been chosen to show the user this Archer notation as well.

2.2.4 Graph

The entire Feature Model is taken and drawn into a graph. First, the Unsatisfiable Core that is generated by the Z3 solver is taken and all its Features

This method also ensures that Features that do not show up in the Unsatisfiable Core still are being shown as problematic Features.



In this view, it becomes immediately clear where the problem in the Feature Model lies in more complex models. The Feature Model on the left is the aforementioned Simple Model where both the Root and the Child Feature are present and the cause of the unsatisfiability, thus the entire model is highlighted in orange and red.

2.3 Generating solutions

7

of, the tool ideally gives options for them to solve their problems. If an end-user clicks on a Feature in the interface then an API request is sent to check which options are viable to change this Feature into a satisfiable Feature again. This is done by obtaining which Feature is selected and taking the following actions:

- **Make Optional:** Take the parent Hierarchical constraint of the current Feature and make it optional. If this results in a satisfiable model add this to the list of possible fixes.
- **Delete Cross-Hierarchy constraints:** Removing the Cross-Hierarchy constraints in the Fast Diagnose result will always result in a satisfiable Feature Model.

Turning the parent Hierarchical constraint of a Feature into an Optional constraint is a method found by the Elfsquad support team that has fixed a lot of their problematic Feature Models. As per their suggestion, I have added this to the list of possible fixes and it seems to be a very reliable method of fixing problematic Feature Models.

If the Feature is **green** it is implied that this Feature does not have anything to do with this model to be unsatisfiable so nothing happens.

If the Feature is **orange** or **red** then we can take action to fix the satisfiability of the Feature Model. The API checks if the Feature is present in the result of the Fast Diagnose algorithm and if so, adds removing its Cross-Hierarchy constraints to the list of solutions.

Next the API takes the parent Hierarchical constraint and turns it into an Optional constraint. Z3 checks if the model is satisfiable now and if so, this is also added to the list of solutions for the Feature.

The options that make the model satisfiable again are returned to the interface and shown to the user so they can take their desired action to change the Feature Model as shown in Figure 2.6:

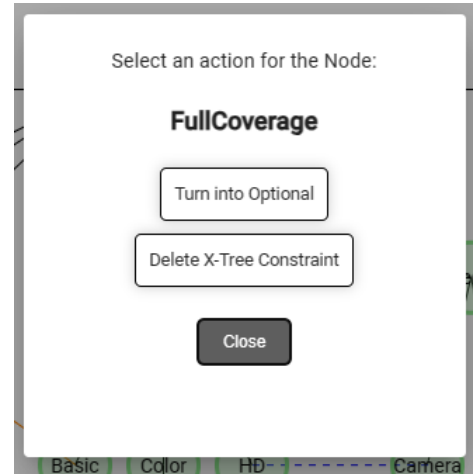


Figure 2.6: Solutions generated by the API to make the Feature Model satisfiable again

2.4 Experiment

In order to test the effectiveness of this tool, 10 Elfsquad employees have given informed consent to participate in this experiment. Each employee will have to diagnose 10 predefined problematic Archer Models (as defined in the appendix A.1).

The participants of this experiment will only see the model definition and not the comments about the solutions. These are checked to see if the answers of the participants were correct or not.

The participants consist of employees of the support and development departments of Elfsquad, which mean these participants do have a lot of technical knowledge about Feature Models and how they are build.

The models are increasing in size, starting with the smallest model and increasing to the biggest and most complex model in order as seen in the appendix A.1. The employees have up to two minutes per model to find where the problem is. This time limit is imposed because the goal of this experiment is to show how much more efficient it is to use the tool and not how well the participants know the Archer modelling language or Feature Models in general. This can be shown in a short amount of time.

The participants of the experiment are informed about the functionality of this tool; what the colours in the graph mean and that they can click on a Feature to see actions they can take. This is

done to get a better representation of how the end-user would use the tool since they would be informed of the same functionality before they start working with the tool.

Since it is in the best interest of the Elfsquad employees to see if the tool has added value to the company, the participants are instructed not to rush their answers and thoroughly look into the problematic models as if they were models from customers requesting their help.

There are two methods of diagnosing each problem. One method is to use the Archer Modeling Language model definition, and the other method will be to use the tool that is created for the Archer Modeling Language for this thesis. Each employee's method of diagnosing the model will be randomized such that they have 5 Archer Language models and 5 graphical visualized models. All models will also have 5 employees diagnosing the model as Archer representation and 5 as graph representation to make sure all model results can be compared to each other correctly. This method allows for a comparison between the time it takes the Elfsquad employees to diagnose the problem within each model in the Archer representation and the graphical representation.

The numbers shown in the data are the average time (rounded to the nearest second) that it took for the employees to find the problem within the given Feature Model. The timing is done by hand with a stopwatch running on a mobile phone. The time stops when the participant correctly removes the problematic constraint(s) from the given model and the model is satisfiable again. After obtaining the correct solution, the participant is asked for their thought process to check that what they did was not a guess but an informed decision. If a constraint is removed that is not a problematic constraint, then the participant will receive feedback as such and the time continues.

This is in accordance with how it would work if the Elfsquad support team would suggest making a change and it not being accepted by their customers. In that case they may have to look for further solutions to the problem.

A T-Test is used to check if there is a statistical difference in the time it takes to fix a model when using the tool and not using the tool.

3 Results

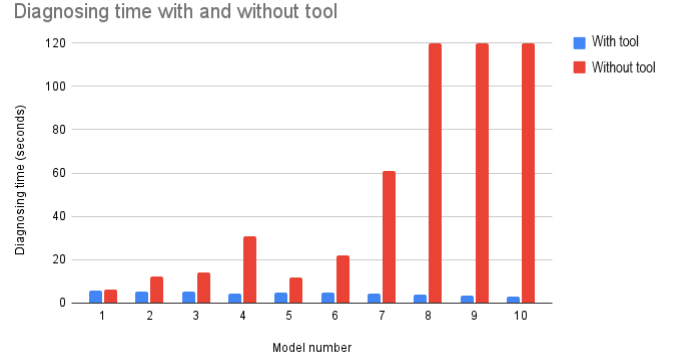


Figure 3.1: Resulting average time for correctly diagnosing each Feature Model with the tool (blue) and without the tool (red)

The graph in Figure 3.1 is the average time it took for each model to be diagnosed and fixed. The blue bars are the time it took with the created tool ($M=4,43$, $SD=0,73$) and the red bars are the time it took without the tool ($M=51,72$, $SD=3,72$). Using the tool to diagnose and fix a Feature Model seems to be significantly faster than not using the tool: $t(98)=9.12$, $p=0.007$.

4 Conclusion

This thesis is trying to find a method to reduce the time spent on diagnosing problems within a Feature Model that does not have a solution. A visualization tool has been created to reduce the time that it will take to find the problematic logic within a Feature Model.

Even with the limitation of only having 120 seconds, the result of the experiment clearly shows that the tool greatly reduces the amount of time that it takes to diagnose a problematic Feature Model ($t(98)=0.12$, $p=0.007$).

For the smaller models this difference is not big enough to completely justify the usage of this tool since the problems within small models can easily be spotted in the textual format of the Archer language.

The real justification for this tool comes with the bigger and more complex models. It took a lot of

time and effort for the participants to find the problem within the model if they did not use the created tool.

The data shows a reduction in the time it took to diagnose and fix the problematic Feature Models number 5 and 6. This is probably because the participants have some basic knowledge of how to fix these models and were looking for the Cross-Hierarchy constraints that they can delete. These two models have a very low amount, resulting in the participants finding the solution faster than anticipated.

The time it took to fix the models using the tool seems to decrease the more the participants used the tool. This is likely because the participants had not seen the tool before but only knew its functionalities. Once the participants worked with the tool and got used to working with it, the time stabilized on a very short amount of time (around 2 to 3 seconds per model).

This tool allows users to more efficiently diagnose and fix problems for Feature Models that are unsatisfiable than having to use the textual input as shown in the appendix (A.1).

5 Critique

With this thesis the visualization tool has only been tested and used by employees of Elfsquad. The ultimate goal is to have the customers of Elfsquad to use the tool instead of the employees. This is not possible yet, as the Archer Modeling Language itself is still in development and set to be released later on. The customers are scheduled to receive access to this tool at the same time as access to the Archer Modeling Language.

Furthermore, as of right now, the newest version of the Archer Modeling Language has not been implemented into the tool yet.

The Archer Language is an ongoing project developed in-house at Elfsquad. Keeping up to date with the newest version of Archer would be too much implementation jumping from version to version that the decision has been made to take an older version of Archer and implement this into the tool.

Once Archer is completed, the tool is operational, and the tool has shown the company that the tool is able to increase the operational workflow for Elfsquad, then the complete version

of Archer can be implemented into the tool for future use.

Another point of critique is that this tool is only tested for relatively small Feature Models. The actual Feature Models used in Elfsquad are many times larger than the ones used in this thesis. There is no way of knowing if these larger models will still be visually easy to debug if the scale becomes that much larger, but the assumption is that it will be. The only way to really know this is to implement this tool into the Elfsquad software and use it with the Feature Models it is intended for.

Something else that could be a point of critique is that the Fast Diagnose implementation currently only looks for the first solution and not the best. This means that there could be a better solution to the given problem, which we do not show to the customer at this point in time.

A big point of critique is that the pop-up that shows the methods to fix the user's Feature Model is a very limited way of fixing these kinds of models. More elaborate methods could be generated, but since this tool is meant as a first-step on which will be built further, it is beyond the scope of this research paper.

6 Future Research

This thesis functions as the basis of a diagnostics tool for the Elfsquad Archer Modeling Language. This means that future research depends on the implementation of the Archer Modeling Language and the ways that Elfsquad's customers want to use the tool. Since Archer is still in development within Elfsquad, there is no saying exactly what the customers will come up with and how they want to use it. This however does not mean that no future research can be thought of:

Interpret the Z3 generated proof.

As of right now, nothing is being done with the Z3-generated proof except for being shown to the end user. This proof does contain all the information

as to why it is not possible to find a solution and perhaps an easier interpretation can be created for diagnosing problematic Feature Models using this proof.

Complete Archer Modeling Language implementation.

For now, there is only a limited component of the Archer Modeling Language implemented in the tool. Ongoing development into Archer has prevented this tool from keeping up with the latest version, as it would be too much work to keep up with all and every change as they come.

Generate more options to fix the problem

The implementation right now is that only two potential methods of fixing the model are taken into account when clicking on a Feature in the graph; making the parent relationship Optional, and removing the Cross-Hierarchy constraint if it exists in the Fast Diagnose result. This could potentially be expanded by more sophisticated actions in the future.

Implement Numeric Domains.

This tool currently only works with Features that make use of Boolean Domains. A Feature can be selected or not. Elfsquad has encountered Feature Models where customers want to make use of Numeric Domains as well. These Numeric Domains are acted upon by different Constraints, ones that are not present in the simple Boolean Feature Models that are discussed in this paper.

In order to support these constraints, new visualizations should be added to the tool in order to give information pertaining to the numeric constraints in the Model.

References

- Benavides, D., Segura, S., & Ruiz-Cortés, A. (2010, 09). Automated analysis of feature models 20 years later: A literature review. *Information Systems*, 35, 615-636. doi: 10.1016/j.is.2010.01.001
- de Moura, L., & Bjørner, N. (2008, 04). Z3: an efficient smt solver. In (Vol. 4963, p. 337-340). doi: 10.1007/978-3-540-78800-3_24
- Hotz, L., Felfernig, A., Stumptner, M., Ryabokon, A., O'Neill, C., & Wolter, K. (2014a, 04). Chapter 6. configuration knowledge representation and reasoning. In (p. 41-72). doi: 10.1016/B978-0-12-415817-7.00006-2
- Hotz, L., Felfernig, A., Stumptner, M., Ryabokon, A., O'Neill, C., & Wolter, K. (2014b, 04). Chapter 7. configuration knowledge representation and reasoning. In (p. 41-72). doi: 10.1016/B978-0-12-415817-7.00006-2
- Lee, K., Kang, K., & Lee, J. (2002, 04). Concepts and guidelines of feature modeling for product line software engineering. In (p. 62-77). doi: 10.1007/3-540-46020-9_5
- Perron, L. (2011, 01). Operations research and constraint programming at google. In (p. 2). doi: 10.1007/978-3-642-23786-7_2
- Prud'homme, C., Fages, J.-G., & Lorca, X. (2019). Choco solver. *Website, March*.
- Rothenhäusler, L. (2022). *d3.js and its potential in data visualization* [bachelorthesis]. doi: 10.25933/opus4-2853
- Russell, S. J., & Norvig, P. (2010). *Artificial intelligence: A modern approach*. Prentice Hall.
- Thüm, T., Kästner, C., Benduhn, F., Meinicke, J., Saake, G., & Leich, T. (2014). Featureide: An extensible framework for feature-oriented software development. *Science of Computer Programming*, 79, 70-85. (Experimental Software and Toolkits (EST 4): A special issue of the Workshop on Academic Software Development Tools and Techniques (WASDeTT-3 2010)) doi: <https://doi.org/10.1016/j.scico.2012.06.002>

A Appendix

A.1 Model definitions

There can be multiple solutions to a model. Each solution is indicated with a different number. If multiple constraints need to be removed to make the Feature Model satisfiable again, then these receive a single number as they are part of the same solution.

A.1.1 Model 1

```
model [Phone] {
    [Phone] - mandatory -> [Screen]
    [Screen] - excludes -> [Screen]
}

# Solutions:
# 1) [Screen] - excludes -> [Screen]
```

A.1.2 Model 2

```
model [Phone] {
    [Phone] - mandatory -> [Screen]
    [Phone] - mandatory -> [FingerPrint]
    [FingerPrint] - requires -> [Screen]
    [Screen] - excludes -> [FingerPrint]
}

# Solutions:
# 1) [Screen] - excludes -> [FingerPrint]
```

A.1.3 Model 3

```
model [Phone] {
    [Phone] - mandatory -> [Screen]
    [Phone] - mandatory -> [FingerPrint]
    [Phone] - alternative -> [GPS]
    [Phone] - alternative -> [Media]
    [Screen] - excludes -> [GPS]
    [FingerPrint] - excludes -> [Media]
}

# Solutions:
# 1) [Screen] - excludes -> [GPS]
#
# 2) [FingerPrint] - excludes -> [Media]
```

A.1.4 Model 4

```
model [Root] {
    [Root] - optional -> [Gravity]
    [Root] - mandatory -> [Screws]
    [Root] - mandatory -> [Distance]
    [Root] - alternative -> [Height]
    [Root] - alternative -> [Width]
    [Root] - alternative -> [Depth]
    [Root] - alternative -> [Scale]
    [Height] - requires -> [Gravity]
    [Distance] - excludes -> [Screws]
    [Gravity] - requires -> [Depth]
    [Screws] - excludes -> [Distance]
}

# Solutions:
# 1) [Screws] - excludes -> [Distance]
#    [Distance] - excludes -> [Screws]
```

A.1.5 Model 5

```
model [Root] {
    [Root] - mandatory -> [Calculation]
    [Root] - mandatory -> [Price]
    [Root] - mandatory -> [Estimations]

    [Calculation] - mandatory -> [Distance]
    [Calculation] - mandatory -> [Height]
    [Calculation] - or -> [Width]
    [Calculation] - or -> [Length]

    [Distance] - alternative -> [Miles]
    [Distance] - alternative -> [Kilometers]
    [Distance] - mandatory -> [Value]
    [Distance] - optional -> [ErrorMargin]

    [Price] - mandatory -> [BillOfMaterial]
    [Price] - mandatory -> [CostOverview]
    [Price] - optional -> [Invoice]
    [Price] - mandatory -> [HoursWorked]

    [Value] - excludes -> [CostOverview]
}

# Solutions:
# 1) [Value] - excludes -> [CostOverview]
```

A.1.6 Model 6

```
model [Phone] {
  [Phone] - mandatory -> [Screen]
  [Phone] - mandatory -> [Finger Print]
  [Phone] - alternative -> [alt1]
  [Phone] - alternative -> [alt2]

  [alt1] - alternative -> [alt11]
  [alt1] - alternative -> [alt12]
  [alt1] - alternative -> [alt13]
  [alt1] - alternative -> [alt14]

  [alt2] - alternative -> [alt21]
  [alt2] - alternative -> [alt22]
  [alt2] - alternative -> [alt23]
  [alt2] - alternative -> [alt24]

  [alt12] - alternative -> [alt121]
  [alt12] - alternative -> [alt122]

  [alt2] - requires-> [alt1]
  [alt12] - excludes-> [alt21]
  [alt1] - requires-> [alt2]
}

# Solutions:
# 1) [alt2] - requires-> [alt1]
#
# 2) [alt1] - requires-> [alt2]
```

A.1.7 Model 7

```
model [Phone] {
  [Phone] - mandatory -> [Calls]
  [Phone] - optional -> [GPS]

  [Phone] - mandatory -> [Warranty]
  [Warranty] - mandatory -> [FullCoverage]

  [Phone] - mandatory -> [Screen]
  [Screen] - alternative -> [Basic]
  [Screen] - alternative -> [Color]
  [Screen] - alternative -> [HD]

  [Color] - mandatory -> [RGB]
  [Calls] - mandatory -> [Wifi]

  [Phone] - optional -> [Media]
  [Media] - or -> [Camera]
```

```
[Media] - or -> [MP3]

[Camera] - requires -> [HD]
[GPS] - excludes -> [Basic]
[Wifi] - excludes -> [FullCoverage]
}
```

```
# Solutions:
# 1) [Wifi] - excludes -> [FullCoverage]
```

A.1.8 Model 8

```
model [Phone] {
  [Media] - or -> [MP3]
  [Phone] - mandatory -> [Screen]
  [Screen] - alternative -> [Basic]
  [Color] - mandatory -> [RGB]
  [Calls] - mandatory -> [Wifi]
  [Phone] - mandatory -> [Calls]
  [Phone] - optional -> [GPS]
  [Phone] - mandatory -> [Warranty]
  [Warranty] - mandatory -> [FullCoverage]
  [Screen] - alternative -> [Color]
  [Camera] - requires -> [HD]
  [Screen] - alternative -> [HD]
  [GPS] - excludes -> [Basic]
  [Phone] - optional -> [Media]
  [Media] - or -> [Camera]
  [Wifi] - excludes -> [FullCoverage]
}
```

```
# Solutions:
# 1) [Wifi] - excludes -> [FullCoverage]
```

A.1.9 Model 9

```
model [Phone] {
  [Phone] - mandatory -> [Screen]
  [Phone] - mandatory -> [FingerPrint]
  [Phone] - alternative -> [Bluetooth]
  [Phone] - alternative -> [GPS]

  [Bluetooth] - alternative -> [V5]
  [Bluetooth] - alternative -> [V4]
  [Bluetooth] - alternative -> [V3]
  [Bluetooth] - alternative -> [V2]
  [Bluetooth] - alternative -> [V1]

  [GPS] - alternative -> [Locally]
  [GPS] - alternative -> [NationWide]
```

```

[GPS] - alternative -> [Continental]
[GPS] - alternative -> [WorldWide]

[V1] - requires-> [WiFi]
[Locally] - requires-> [V2]
[V3] - requires-> [NationWide]
[Bluetooth] - requires-> [GPS]
[Continental] - requires-> [V4]
[V5] - requires-> [WorldWide]
[GPS] - requires-> [Bluetooth]
}

# Solutions:
# 1) [GPS] - requires-> [Bluetooth]
#
# 2) [Bluetooth] - requires-> [GPS]

# Solutions:
# 1) [Bluetooth] - requires -> [Antenna]

# 2) [GPS] - requires -> [Antenna]

# 3) [Antenna] - requires -> [V5]

```

A.1.10 Model 10

```

model [Phone] {
    [Phone] - mandatory -> [Screen]
    [Phone] - mandatory -> [FingerPrint]
    [Phone] - optional -> [WiFi]
    [Phone] - alternative -> [Bluetooth]
    [Phone] - alternative -> [GPS]
    [Phone] - optional -> [Antenna]

    [Bluetooth] - alternative -> [V5]
    [Bluetooth] - alternative -> [V4]
    [Bluetooth] - alternative -> [V3]
    [Bluetooth] - alternative -> [V2]
    [Bluetooth] - alternative -> [V1]

    [V1] - requires-> [WiFi]
    [V2] - requires-> [Locally]
    [V3] - requires-> [NationWide]
    [V4] - requires-> [Continental]
    [V5] - requires-> [WorldWide]

    [GPS] - alternative -> [Locally]
    [GPS] - alternative -> [NationWide]
    [GPS] - alternative -> [Continental]
    [GPS] - alternative -> [WorldWide]

    [WiFi] - Excludes-> [GPS]
    [Bluetooth] - requires -> [Antenna]
    [GPS] - requires -> [Antenna]
    [Antenna] - requires -> [V5]
    [WorldWide] - requires -> [WiFi]
}

```

A.2 Proof results

A.2.1 Proof Model 1

```

model [Phone] {
  [Phone] - mandatory -> [Screen]
  [Screen] - excludes -> [Screen]
}

(let ((a!1 (def-axiom (or (not (= Phone Screen)) (not Phone) Screen)))
      (a!2 (monotonicity (rewrite (= (= Phone true) Phone))
                          (= (= proxy!2 (= Phone true)) (= proxy!2 Phone))))
      (a!3 (rewrite (= (= proxy!2 Phone) (or Phone (not proxy!2))))
      (a!6 (asserted (= proxy!1 (or (not Screen) (not Screen))))
      (a!7 (rewrite (= (or (not Screen) (not Screen)) (not Screen)))
      (a!8 (= (= proxy!1 (or (not Screen) (not Screen)))
              (= proxy!1 (not Screen))))
      (a!9 (rewrite (= (= proxy!1 (not Screen))
                      (or (not Screen) (not proxy!1))))
      (a!10 (= (= proxy!1 (or (not Screen) (not Screen)))
              (or (not Screen) (not proxy!1))))
      (a!12 (rewrite (= (= proxy!0 (= Phone Screen))
                      (or (not proxy!0) (= Phone Screen))))))
  (let ((a!4 (trans a!2
                    a!3
                    (= (= proxy!2 (= Phone true)) (or Phone (not proxy!2))))
        (a!11 (unit-resolution (mp a!6
                                   (trans (monotonicity a!7 a!8) a!9 a!10)
                                   (or (not Screen) (not proxy!1)))
                              (asserted proxy!1)
                              (not Screen)))
        (a!13 (mp (asserted (= proxy!0 (= Phone Screen)))
                  a!12
                  (or (not proxy!0) (= Phone Screen))))
        (let ((a!5 (mp (asserted (= proxy!2 (= Phone true)))
                      a!4
                      (or Phone (not proxy!2))))
              (unit-resolution a!1
                              (unit-resolution a!5 (asserted proxy!2) Phone)
                              a!11
                              (unit-resolution a!13 (asserted proxy!0) (= Phone Screen)
                                                  false))))))

```

Figure A.1: Z3 generated proof of Model 1

A.2.2 Proof Model 2

```

model [Phone] {
  [Phone] - mandatory -> [Screen]
  [Phone] - mandatory -> [FingerPrint]
  [FingerPrint] - requires -> [Screen]
  [Screen] - excludes -> [FingerPrint]
}

(let ((a!1 (asserted (= proxy!1 (or (not Screen) (not FingerPrint))))
      (a!2 (rewrite (= (or (not Screen) (not FingerPrint))
                      (or (not Screen) (not FingerPrint))))
      (a!3 (= (= proxy!1 (or (not Screen) (not FingerPrint)))
              (= proxy!1 (or (not Screen) (not FingerPrint))))
      (a!4 (= (= proxy!1 (or (not Screen) (not FingerPrint))
                      (or (not Screen) (not FingerPrint) (not proxy!1))))
      (a!5 (rewrite (= (or (not Screen) (not FingerPrint) (not proxy!1))
                      (or (not Screen) (not FingerPrint) (not proxy!1))))
      (a!7 (= (or (not Screen) (not FingerPrint) (not proxy!1))
              (or (or (not Screen) (not FingerPrint)) (not proxy!1))))
      (a!9 (= (or (or (not Screen) (not FingerPrint)) (not proxy!1))
              (or (or (not Screen) (not FingerPrint)) (not proxy!1))))
      (a!10 (= (or (or (not Screen) (not FingerPrint)) (not proxy!1))
              (or (not Screen) (not FingerPrint) (not proxy!1))))
      (a!11 (def-axiom (or (not (= Phone FingerPrint)) (not Phone) FingerPrint)))
      (a!12 (monotonicity (rewrite (= (= Phone true) Phone))
                          (= (= proxy!4 (= Phone true)) (= proxy!4 Phone))))
      (a!13 (rewrite (= (= proxy!4 Phone) (or Phone (not proxy!4))))
      (a!16 (rewrite (= (= proxy!2 (= Phone FingerPrint))
                      (or (not proxy!2) (= Phone FingerPrint))))
      (a!18 (def-axiom (or (not (= Phone Screen)) (not Phone) Screen)))
      (a!19 (rewrite (= (= proxy!0 (= Phone Screen))
                      (or (not proxy!0) (= Phone Screen))))))
  (let ((a!6 (mp (mp a!1
                    (trans (monotonicity a!2 a!3) (rewrite a!4) a!4)
                    (or (not Screen) (not FingerPrint) (not proxy!1))))
        a!5
        (or (not Screen) (not FingerPrint) (not proxy!1))))
        (a!14 (trans a!12
                     a!13
                     (= (= proxy!4 (= Phone true)) (or Phone (not proxy!4))))
        (a!17 (mp (asserted (= proxy!2 (= Phone FingerPrint)))
                  a!16
                  (or (not proxy!2) (= Phone FingerPrint))))
        (a!20 (mp (asserted (= proxy!0 (= Phone Screen)))
                  a!19
                  (or (not proxy!0) (= Phone Screen))))))
    (let ((a!8 (mp a!6
                  (rewrite a!7
                    (or (or (not Screen) (not FingerPrint)) (not proxy!1))))
          (a!15 (mp (asserted (= proxy!4 (= Phone true)))
                    a!14
                    (or Phone (not proxy!4))))
          (unit-resolution (mp a!8
                              (trans (monotonicity a!2 a!9) (rewrite a!10) a!10)
                              (or (not Screen) (not FingerPrint) (not proxy!1)))
                          (unit-resolution a!11
                              (unit-resolution a!15
                                  (asserted proxy!4)
                                  Phone)
                              (unit-resolution a!17
                                  (asserted proxy!2)
                                  (= Phone FingerPrint))
                              FingerPrint)
                          (unit-resolution a!18
                              (unit-resolution a!15
                                  (asserted proxy!4)
                                  Phone)
                              (unit-resolution a!17
                                  (asserted proxy!2)
                                  (= Phone FingerPrint))
                              FingerPrint)
                          (unit-resolution a!18
                              (unit-resolution a!15
                                  (asserted proxy!4)
                                  Phone)
                              (unit-resolution a!20
                                  (asserted proxy!0)
                                  (= Phone Screen))
                              Screen)
                          (asserted proxy!1)
                          false))))))

```

Figure A.2: Z3 generated proof of Model 2

A.2.3 Proof Model 3

```

model [Phone] {
  [Phone] - mandatory -> [Screen]
  [Phone] - mandatory -> [FingerPrint]
  [Phone] - alternative -> [GPS]
  [Phone] - alternative -> [Media]
  [Screen] - excludes -> [GPS]
  [FingerPrint] - excludes -> [Media]
}

(
  (let ((a1 (def-axiom (or (not (or GPS Media)) GPS Media))))
    (a2 (asserted (= proxy3 (or (not FingerPrint) (not Media)))))
    (a3 (= (= proxy3 (or (not FingerPrint) (not Media)))))
    (or (not FingerPrint) (not Media) (not proxy3)))
    (a4 (= (or (not FingerPrint) (not Media) (not proxy3)))
    (or (or (not FingerPrint) (not Media)) (not proxy3)))
    (a6 (= (or (not FingerPrint) (not Media) (not proxy3)))
    (or (not FingerPrint) (not Media) (not proxy3)))
    (a7 (def-axiom (or (not (= Phone FingerPrint)) (not Phone) FingerPrint)))
    (a8 (monotonicity (rewrite (= (= Phone true) Phone))
    (= (= proxy5 (= Phone true)) (= proxy5 Phone))))
    (a9 (rewrite (= (= proxy5 Phone) (or Phone (not proxy5)))))
    (a12 (rewrite (= (= proxy2 (= Phone FingerPrint))
    (or (not proxy2) (= Phone FingerPrint))))
    (a15 (asserted (= proxy1 (or (not Screen) (not GPS)))))
    (a16 (rewrite (= (or (not Screen) (not GPS)) (or (not Screen) (not GPS)))))
    (a17 (= (= proxy1 (or (not Screen) (not GPS)))))
    (= proxy1 (or (not Screen) (not GPS))))
    (a18 (= (= proxy1 (or (not Screen) (not GPS)))))
    (or (not Screen) (not GPS) (not proxy1)))
    (a19 (rewrite (= (or (not Screen) (not GPS) (not proxy1))
    (or (not Screen) (not GPS) (not proxy1)))))
    (a21 (= (or (not Screen) (not GPS) (not proxy1))
    (or (or (not Screen) (not GPS)) (not proxy1))))
    (a23 (= (or (or (not Screen) (not GPS)) (not proxy1))
    (or (or (not Screen) (not GPS)) (not proxy1))))
    (a24 (= (or (or (not Screen) (not GPS)) (not proxy1))
    (or (not Screen) (not GPS) (not proxy1))))
    (a25 (def-axiom (or (not (= Phone Screen)) (not Phone) Screen)))
    (a26 (rewrite (= (= proxy0 (= Phone Screen))
    (or (not proxy0) (= Phone Screen))))
    (a29 (or (not (= Phone (or GPS Media)) (not Phone) (or GPS Media))))
    (a30 (or (not (= Phone (or GPS Media)) (not (or (not GPS) (not Media)))))
    (a32 (= proxy4
    (and (= Phone (or GPS Media)) ([_ at-most 1] GPS Media))))
    (a33 (and (= Phone (or GPS Media)) (or (not GPS) (not Media))))
  (let ((a15 (mp (mp a12
    (rewrite a13)
    (or (not FingerPrint) (not Media) (not proxy3)))
    (rewrite a14)
    (or (or (not FingerPrint) (not Media)) (not proxy3))))
    (a10 (trans a18
    a19
    (= (= proxy5 (= Phone true)) (or Phone (not proxy5)))))
    (a13 (mp (asserted (= proxy2 (= Phone FingerPrint)))
    a12
    (or (not proxy2) (= Phone FingerPrint))))
    (a20 (mp (mp a15
    (trans (monotonicity a16 a17) (rewrite a18) a18)
    (or (not Screen) (not GPS) (not proxy1)))
    a19
    (or (not Screen) (not GPS) (not proxy1))))
    (a27 (mp (asserted (= proxy0 (= Phone Screen))
    a26
    (or (not proxy0) (= Phone Screen))))
    (a31 (def-axiom (or a30 (= Phone (or GPS Media)))))
    (a34 (= (and (= Phone (or GPS Media)) ([_ at-most 1] GPS Media)) a33))
    (a35 (rewrite (= (= proxy4 a33) (or (not proxy4) a33)))
    (a37 (rewrite (= (or (not proxy4) a33) (or (not proxy4) (not a30)))))
  (let ((a11 (mp (asserted (= proxy5 (= Phone true))
    a10
    (or Phone (not proxy5)))))
    (a22 (mp a120
    (rewrite a21)
    (or (or (not Screen) (not GPS)) (not proxy1))))
    (a36 (trans (monotonicity (rewrite a34) (= a32 (= proxy4 a33)))
    a35
    (= a32 (or (not proxy4) a33))))
  (let ((a14 (unit-resolution (mp a5
    (rewrite a16)
    (or (not FingerPrint)
    (not Media)
    (not proxy3)))
    (unit-resolution a17
    (unit-resolution a11
    (asserted proxy5)
    Phone)
    (unit-resolution a13
    (asserted proxy2)
    (= Phone
    FingerPrint))
    FingerPrint)
    (asserted proxy3)
    (not Media))))
    (a28 (unit-resolution (mp a22
    (trans (monotonicity a16 a23)
    (rewrite a24)
    a24)
    (or (not Screen) (not GPS) (not proxy1)))
    (unit-resolution a25
    (unit-resolution a11
    (asserted proxy5)
    Phone)
    (unit-resolution a27
    (asserted proxy0)
    (= Phone Screen))
    Screen)
    (asserted proxy1)
    (not GPS)))
    (a38 (mp (mp (asserted a32) a36 (or (not proxy4) a33))
    a37
    (or (not proxy4) (not a30))))
  (let ((a19 (unit-resolution (def-axiom a29)
    (unit-resolution a11 (asserted proxy5) Phone)
    (unit-resolution a31
    (unit-resolution a38
    (asserted proxy4)
    (not a30))
    (= Phone (or GPS Media)))
    (or GPS Media))))
    (unit-resolution a11 a28 a39 false))))))

```

Figure A.3: Z3 generated proof of Model 3